

# Q2 PROCUREMENT DEDUPLICATION

MinHash, Locality-Sensitive Hashing, and PostgreSQL Evaluation

## **Final Project Report**

Dataset: Procurement notices and labelled pairs

Total notices: 12,000

Total portals: 260

Method: Character 5-gram MinHash with LSH

Database: PostgreSQL 16 using Docker

## 1. Introduction

Procurement systems may contain repeated or near-duplicate notices published through different portals or at different times. Detecting these records can reduce duplicate results, improve search quality, and support analysis.

This project implements and evaluates a candidate-retrieval approach for procurement notice deduplication. The approach represents notice text using character n-grams, creates MinHash signatures, uses Locality-Sensitive Hashing (LSH) to retrieve possible matches, and evaluates the results using labelled pairs. The data is also stored in PostgreSQL, where query plans and index usage are examined.

## 2. Objectives

- Profile the procurement notice dataset.
- Represent notice text for similarity comparison.
- Use MinHash and LSH to retrieve possible duplicate notices.
- Evaluate retrieval using labelled same and different pairs.
- Study the effect of different LSH thresholds.
- Load the data into PostgreSQL and inspect query performance.

## 3. Project Structure

| File/Folder                   | Purpose                                             |
|-------------------------------|-----------------------------------------------------|
| data/data_2/                  | Input notices, labelled pairs, and supporting data. |
| src/profile_data.py           | Basic data profiling and checks.                    |
| src/corpus_profile.py         | Corpus-level statistics.                            |
| src/retrieval_experiment.py   | Baseline MinHash and LSH experiment.                |
| src/threshold_sweep.py        | Evaluation across LSH thresholds.                   |
| src/plot_threshold_sweep.py   | Generation of threshold plots.                      |
| src/load_postgres.py          | Loading data into PostgreSQL.                       |
| sql/schema.sql                | Database tables and indexes.                        |
| sql/performance.sql           | EXPLAIN ANALYZE queries.                            |
| outputs/                      | CSV, JSON, and plot outputs.                        |
| reports/experiment_results.md | Experiment notes and results.                       |

## 4. Dataset Profile

The corpus contains procurement notices collected from multiple portals. The labelled\_pairs.csv file was used for evaluation. The \_truth directory was not used for evaluation.

| Property                 | Result                   |
|--------------------------|--------------------------|
| Total notices            | 12,000                   |
| Total columns            | 7                        |
| Unique notice IDs        | 12,000                   |
| Duplicate notice IDs     | 0                        |
| Unique portals           | 260                      |
| Labelled pairs           | 900                      |
| Same-labelled pairs      | 279                      |
| Different-labelled pairs | 621                      |
| Published date range     | 2024-01-01 to 2025-11-27 |
| Closing date range       | 2024-01-16 to 2025-12-08 |

The seven columns are:

- notice\_id
- portal\_id
- published\_at
- title
- body
- estimated\_value
- closing\_date

#### 4.1 Text Statistics

| Field         | Mean     | Median  | Minimum | Maximum |
|---------------|----------|---------|---------|---------|
| Title length  | 80.57    | 80      | 40      | 131     |
| Body length   | 4,527.30 | 4,482   | 1,500   | 8,127   |
| Combined text | 4,608.87 | 4,563.5 | 1,543   | 8,224   |

All seven columns had zero missing values. Estimated values ranged from approximately 1,540,000 to 999,750,000. The mean estimated value was approximately 133,481,100 and the median was approximately 37,540,000.

The notices were distributed across 260 portals. The most frequent portals included P094 with 1,426 notices, P002 with 800, P006 with 792, P001 with 778, P003 with 772, P005 with 768, and P004 with 755.

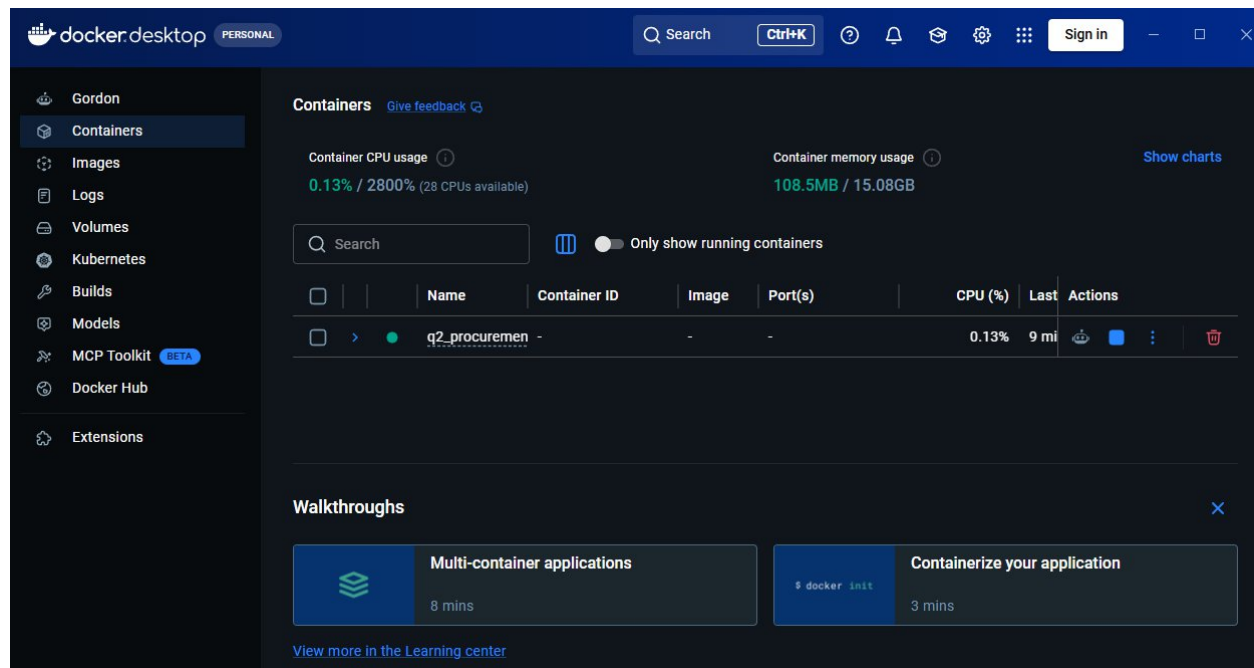


Figure 1. Docker Desktop showing the running Q2 procurement project.

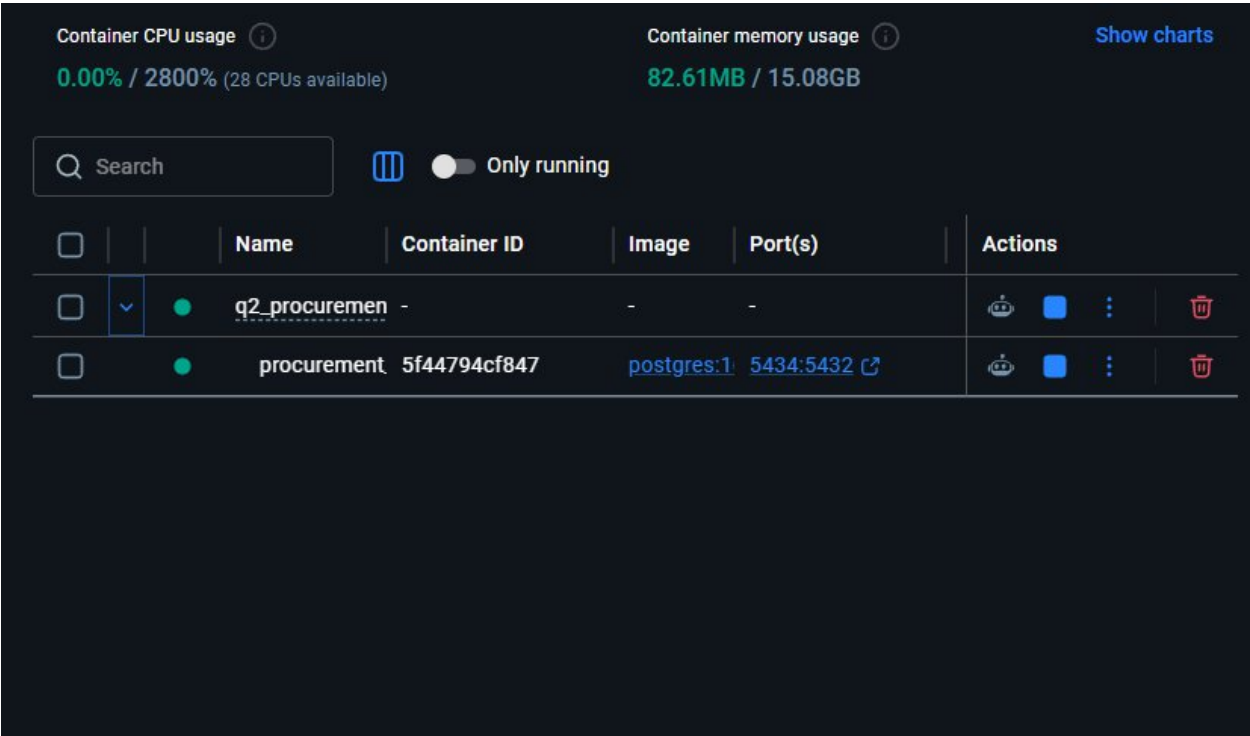


Figure 2. PostgreSQL container running with port 5434 mapped to 5432.

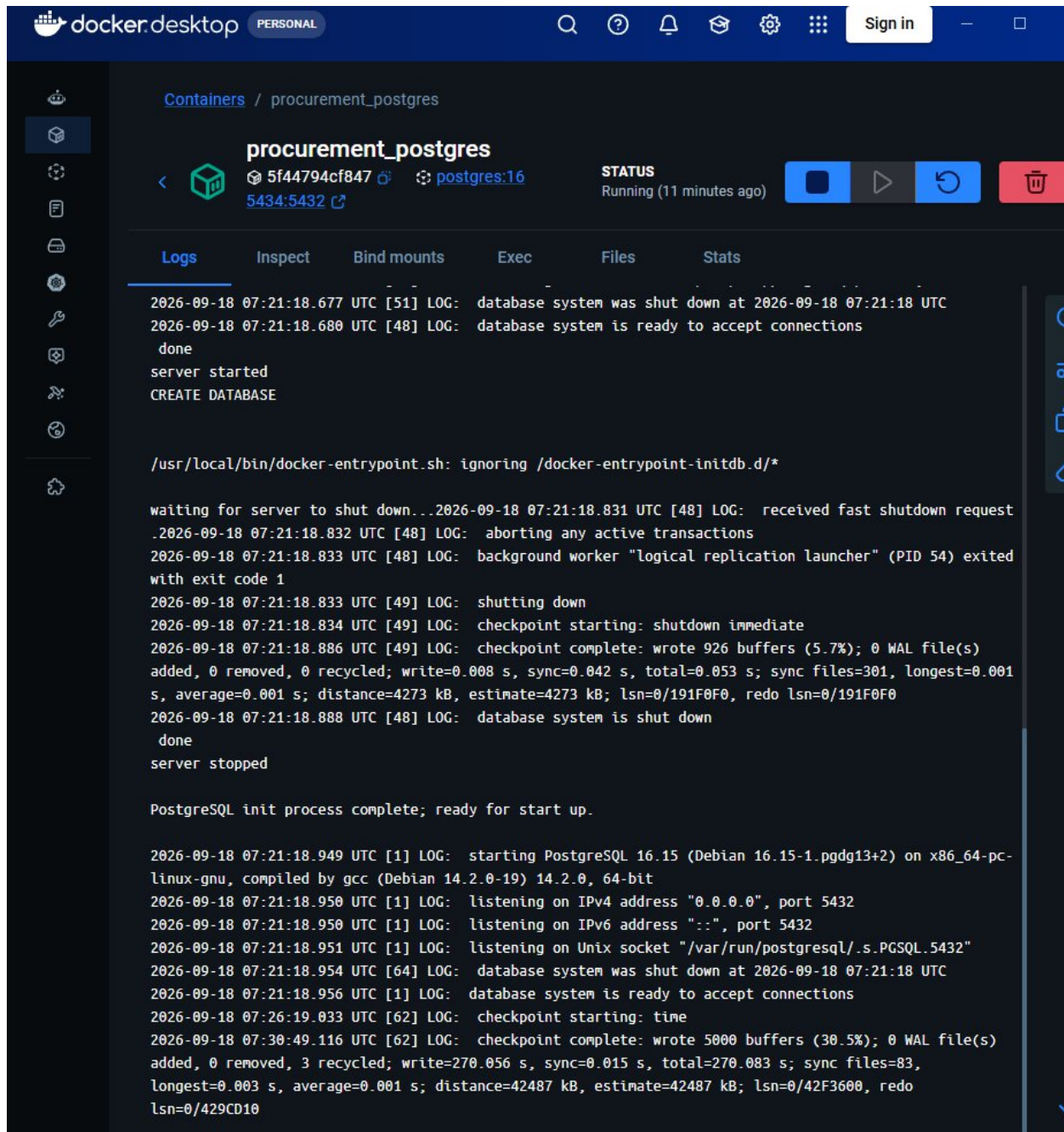


Figure 3. PostgreSQL logs showing that the database is ready to accept connections.

## 5. Methodology

### 5.1 Text Representation

Each notice was represented using character 5-grams generated from the combined title and body text. Character n-grams can capture partial word patterns and tolerate small changes in spelling, punctuation, formatting, and wording.

## 5.2 MinHash Signatures

MinHash was used to create compact signatures for the text representations. The experiment used 128 permutations. MinHash approximates Jaccard similarity while reducing the cost of comparing full text representations.

## 5.3 Locality-Sensitive Hashing

The MinHash signatures were inserted into an LSH index. For each labelled query notice, the index returned candidate notices whose estimated similarity met the selected threshold.

## 5.4 Evaluation Measures

| Measure                 | Meaning                                                      |
|-------------------------|--------------------------------------------------------------|
| Same-pair recall        | Fraction of labelled same pairs that survive retrieval.      |
| Different-pair survival | Fraction of labelled different pairs that survive retrieval. |
| Mean candidate count    | Average number of candidates returned for a query.           |
| Candidate reduction     | Reduction compared with the full 12,000-notice corpus.       |

## 6. Baseline Retrieval Experiment

The baseline used character 5-gram representations, 128-permutation MinHash signatures, and an LSH threshold of 0.50. Signature creation took approximately 172.35 seconds in the recorded run.

| Metric                           | Result   |
|----------------------------------|----------|
| Total notices                    | 12,000   |
| Labelled pairs evaluated         | 900      |
| Different-labelled pairs         | 621      |
| Same-labelled pairs              | 279      |
| Different-pair survival          | 56.84%   |
| Same-pair recall                 | 77.42%   |
| Mean candidate count             | 4,492.37 |
| Median candidate count           | 4,501.5  |
| Minimum candidate count          | 406      |
| Maximum candidate count          | 8,523    |
| Mean similarity: different pairs | 0.516506 |
| Mean similarity: same pairs      | 0.687752 |

```

CREATE INDEX
NOTICE: relation "idx_notices_published_at" already exists, skipping
CREATE INDEX
CREATE INDEX
NOTICE: relation "idx_notices_closing_date" already exists, skipping
CREATE INDEX
NOTICE: relation "idx_candidates_query_notice" already exists, skipping
CREATE INDEX
NOTICE: relation "idx_candidates_candidate_notice" already exists, skipping
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> python .\src\load_postgres.py
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> py .\src\load_postgres.py
Postals loaded: 260
Notices loaded: 12900
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> py .\src\retrieval_experiment.py
Loading notices...
Notices loaded: 12900
Creating MinHash signatures...
Signature creation time: 172.35 seconds
Building LSH index...
LSH index built.
Evaluating labelled pairs...

Labelled-pair retrieval results:
      count  sum    mean
label
different   621  353  0.568438
same        279   216  0.774194

Candidate count statistics:
count      900.000000
mean      4492.368889
std       1904.726902
min        406.000000
25%       2681.000000
50%       4501.500000
75%       6175.500000
max       8523.000000
Name: candidate_count, dtype: float64

Similarity by label:
      count    mean    std    min    25%    50%    75%    max
label
different   621.0  0.516506  0.155529  0.148438  0.382812  0.523438  0.648625  0.859375
same        279.0  0.687752  0.238715  0.195312  0.496094  0.656250  0.945312  1.000000

Saved:
outputs/labelled_pair_results.csv
outputs/retrieval_summary.json
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> |

```

Figure 4. Baseline MinHash and LSH retrieval results.

## 6.1 Interpretation

At threshold 0.50, the system retrieved 77.42% of labelled same pairs. However, 56.84% of labelled different pairs also survived. This indicates that the retrieval stage produces a broad candidate set and may require a later verification or classification stage.

Same-labelled pairs had a higher mean similarity than different-labelled pairs, but the similarity ranges overlap. Therefore, similarity does not perfectly separate the two classes.

## 7. Threshold Sweep

Six LSH thresholds from 0.3 to 0.8 were tested to examine the relationship between recall, different-pair survival, and candidate volume.

| Threshold | Same Recall | Different Survival | Mean Candidates | Candidate Reduction |
|-----------|-------------|--------------------|-----------------|---------------------|
| 0.3       | 0.971326    | 0.864734           | 10,334.86       | 13.88%              |
| 0.4       | 0.888889    | 0.690821           | 7,475.35        | 37.71%              |
| 0.5       | 0.774194    | 0.568438           | 4,492.37        | 62.56%              |
| 0.6       | 0.541219    | 0.283414           | 1,758.32        | 85.35%              |
| 0.7       | 0.390681    | 0.117552           | 682.79          | 94.31%              |
| 0.8       | 0.351254    | 0.037037           | 134.82          | 98.88%              |

```

outputs/labelled_pair_results.csv
outputs/retrieval_summary.json
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> py .\src\threshold_sweep.py
Loading notices...
Creating signatures...
Signature time: 173.92 seconds

Testing threshold: 0.3
Same recall: 0.9713
Different survival: 0.8647
Mean candidates: 10334.86
Median candidates: 10707.0
Evaluation time: 1.72 seconds

Testing threshold: 0.4
Same recall: 0.8889
Different survival: 0.6908
Mean candidates: 7475.35
Median candidates: 7215.0
Evaluation time: 0.96 seconds

Testing threshold: 0.5
Same recall: 0.7742
Different survival: 0.5684
Mean candidates: 4492.37
Median candidates: 4501.5
Evaluation time: 0.52 seconds

Testing threshold: 0.6
Same recall: 0.5412
Different survival: 0.2834
Mean candidates: 1758.32
Median candidates: 1699.5
Evaluation time: 0.2 seconds

Testing threshold: 0.7
Same recall: 0.3907
Different survival: 0.1176
Mean candidates: 682.79
Median candidates: 658.5
Evaluation time: 0.09 seconds

Testing threshold: 0.8
Same recall: 0.3513
Different survival: 0.037
Mean candidates: 134.82
Median candidates: 96.0
Evaluation time: 0.04 seconds

Saved outputs/threshold_sweep.csv
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> |

```

Figure 5. Threshold sweep output from the experiment script.

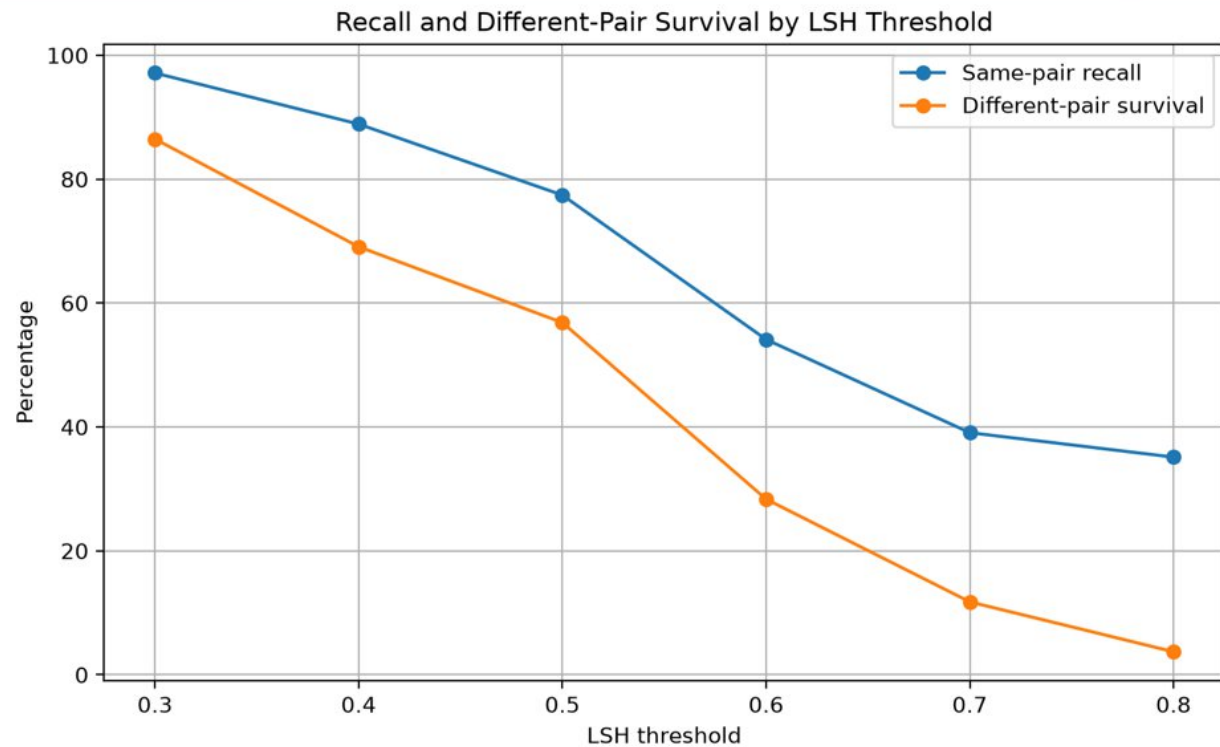


Figure 6. Same-pair recall and different-pair survival across LSH thresholds.

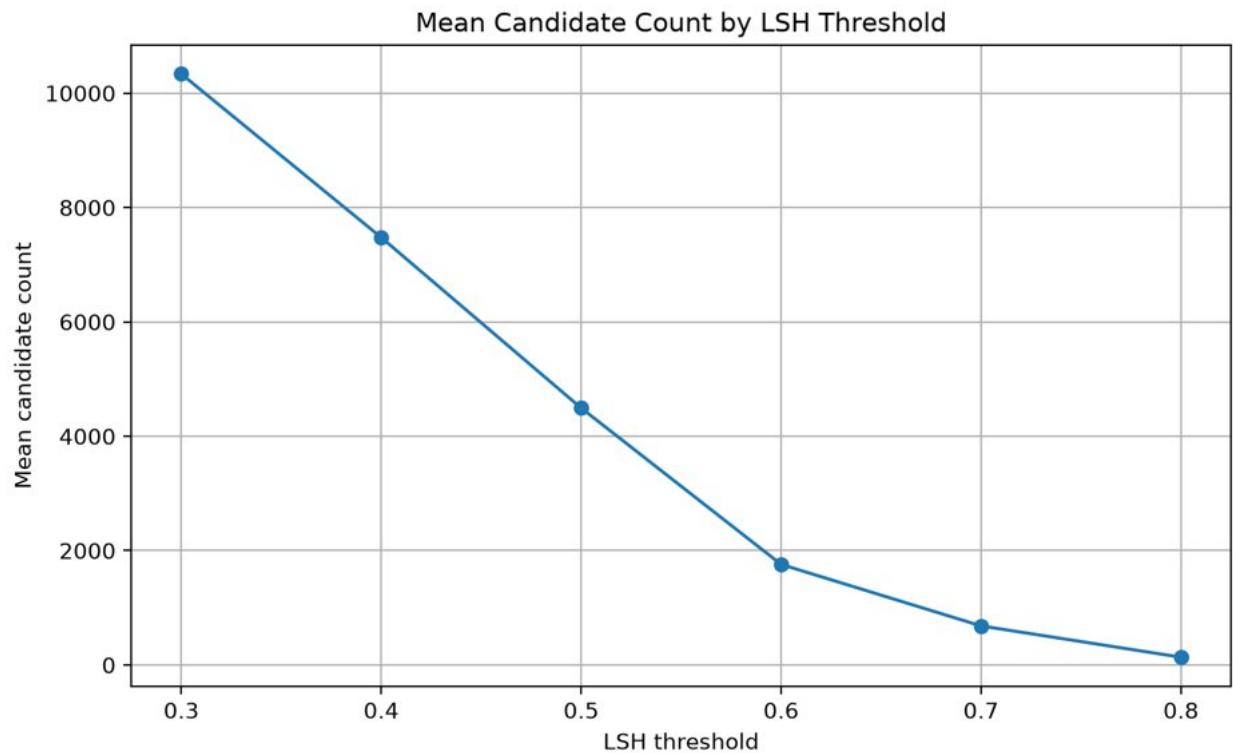


Figure 7. Mean candidate count across LSH thresholds.

### 7.1 Threshold Analysis

At threshold 0.3, same-pair recall was 97.13%, but different-pair survival was 86.47% and the mean candidate count was 10,334.86. At threshold 0.8, the mean candidate count fell to 134.82 and different-pair survival fell to 3.70%, but same-pair recall also fell to 35.13%.

The results show a trade-off: higher thresholds reduce candidate volume and unrelated matches, but they also increase the chance of missing true duplicate pairs. The threshold should therefore depend on the application's recall requirements and verification resources.

## 8. PostgreSQL Implementation

PostgreSQL 16 was deployed using Docker. The database used the procurement database and mapped host port 5434 to PostgreSQL container port 5432.

The schema contains these tables:

- portals
- notices
- notice\_signatures
- retrieval\_runs
- retrieval\_candidates

The schema includes indexes for portal filtering, date filtering, and candidate lookups:

- idx\_notices\_portal\_id
- idx\_notices\_published\_at
- idx\_notices\_closing\_date

- idx\_candidates\_query\_notice
- idx\_candidates\_candidate\_notice

```
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> Get-Content .\sql\schema.sql | docker exec -i procurement_postgres psql -U postgres -d procurement
NOTICE: relation "portals" already exists, skipping
CREATE TABLE
NOTICE: relation "notices" already exists, skipping
CREATE TABLE
CREATE TABLE
NOTICE: relation "notice_signatures" already exists, skipping
NOTICE: relation "retrieval_runs" already exists, skipping
CREATE TABLE
NOTICE: relation "retrieval_candidates" already exists, skipping
CREATE TABLE
NOTICE: relation "idx_notices_portal_id" already exists, skipping
CREATE INDEX
NOTICE: relation "idx_notices_published_at" already exists, skipping
CREATE INDEX
CREATE INDEX
NOTICE: relation "idx_notices_closing_date" already exists, skipping
CREATE INDEX
NOTICE: relation "idx_candidates_query_notice" already exists, skipping
CREATE INDEX
NOTICE: relation "idx_candidates_candidate_notice" already exists, skipping
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> |
```

Figure 8. PostgreSQL schema and index initialization output.

```
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> py .\src\load_postgres.py
Portals loaded: 260
Notices loaded: 12000
PS C:\Users\ub02-glab-012\Q2_Procurement_Deduplication> |
```

Figure 9. Successful loading of 260 portals and 12,000 notices into PostgreSQL.

## 9. PostgreSQL Query Performance

EXPLAIN ANALYZE was used to inspect four representative queries and identify whether PostgreSQL used indexes or sequential scans.

| Query                   | Execution Plan                       | Execution Time |
|-------------------------|--------------------------------------|----------------|
| Filter by portal_id     | Bitmap Index Scan + Bitmap Heap Scan | 0.894 ms       |
| Filter by published_at  | Sequential Scan                      | 0.870 ms       |
| Group by portal_id      | Index Only Scan + GroupAggregate     | 1.036 ms       |
| Body search using ILIKE | Sequential Scan                      | 254.318 ms     |

```

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug procurement\_postgres
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\ub82-glab-012\Q2_Procurement_Deduplication> Get-Content .\sql\performance.sql | docker exec -i procurement_postgres psql -U postgres -d procurement

QUERY PLAN
-----
Bitmap Heap Scan on notices (cost=23.34..400.16 rows=1426 width=206) (actual time=0.081..0.842 rows=1426 loops=1)
  Recheck Cond: ((portal_id)::text = 'P094')::text)
  Heap Blocks: exact=355
  -> Bitmap Index Scan on idx_notices_portal_id (cost=0.00..22.98 rows=1426 width=0) (actual time=0.057..0.058 rows=1426 loops=1)
    Index Cond: ((portal_id)::text = 'P094')::text)
Planning Time: 0.257 ms
Execution Time: 0.894 ms
(7 rows)

QUERY PLAN
-----
Seq Scan on notices (cost=0.00..509.00 rows=5382 width=206) (actual time=0.003..0.768 rows=5377 loops=1)
  Filter: (published_at >= '2025-01-01 00:00:00')::timestamp without time zone)
  Rows Removed by Filter: 6623
Planning Time: 0.035 ms
Execution Time: 0.870 ms
(5 rows)

QUERY PLAN
-----
GroupAggregate (cost=0.29..310.88 rows=260 width=13) (actual time=0.076..1.021 rows=260 loops=1)
  Group Key: portal_id
  -> Index Only Scan using idx_notices_portal_id on notices (cost=0.29..248.28 rows=12000 width=5) (actual time=0.015..0.489 rows=12000 loops=1)
    Heap Fetches: 0
Planning Time: 0.050 ms
Execution Time: 1.036 ms
(6 rows)

QUERY PLAN
-----
Seq Scan on notices (cost=0.00..509.00 rows=96 width=89) (actual time=0.079..253.899 rows=12000 loops=1)
  Filter: (body ~* '%road%')::text)
Planning Time: 0.022 ms
Execution Time: 254.318 ms
(4 rows)

PS C:\Users\ub82-glab-012\Q2_Procurement_Deduplication> |

```

Figure 10. PostgreSQL EXPLAIN ANALYZE results for the four performance queries.

## 9.1 Performance Discussion

The portal filter used `idx_notices_portal_id` through a Bitmap Index Scan and Bitmap Heap Scan, showing that the index supports selective portal filtering.

The published-date query used a sequential scan and returned 5,377 rows while filtering out 6,623 rows. The optimizer may choose a sequential scan when a large portion of the table is relevant or when it estimates that scanning the table is cheaper.

The grouping query used an Index Only Scan followed by GroupAggregate. It processed 12,000 rows and produced 260 portal groups.

The body search used a sequential scan and took 254.318 ms. The condition `ILIKE '%road%'` has a leading wildcard, so a normal B-tree index is not suitable for this contains search. Full-text search or a trigram index could be considered if this type of query is frequent.

## 10. Limitations

- The evaluation uses 900 labelled pairs and does not represent every possible pair in the corpus.
- MinHash and LSH provide approximate retrieval and may miss true matches depending on the threshold and signature configuration.
- The retrieval stage generates candidates; it is not a final duplicate classifier.
- Same and different similarity distributions overlap.
- Execution times depend on the local machine, Docker environment, cache state, and database state.

## 11. Conclusion

The project implemented a procurement notice deduplication pipeline using character 5-gram text representation, 128-permutation MinHash signatures, and LSH candidate retrieval. The corpus contained 12,000 notices from 260 portals, with 900 labelled pairs used for evaluation.

The baseline threshold of 0.50 achieved 77.42% same-pair recall and produced an average of 4,492.37 candidates. The threshold sweep showed that increasing the threshold reduces candidate volume but also reduces same-pair recall. This confirms the trade-off between retaining possible duplicates and reducing unnecessary comparisons.

The PostgreSQL implementation successfully stored the portal and notice data. Query-plan analysis showed index usage for portal filtering and grouping, while the broad date query and wildcard body search used sequential scans.

## 12. Reproducibility Commands

```
docker compose up -d
```

```
py .\src\load_postgres.py
```

```
py .\src\retrieval_experiment.py
```

```
py .\src\threshold_sweep.py
```

```
start .\outputs\threshold_recall_plot.png
```

```
start .\outputs\threshold_candidate_plot.png
```

```
Get-Content .\sql\performance.sql | docker exec -i procurement_postgres psql -U postgres -d procurement
```

## 13. AI Usage Disclosure

AI assistance was used to organize the report, explain technical concepts, improve wording, structure the documentation, and format the project evidence. The dataset values, experiment results, screenshots, code execution outputs, and PostgreSQL performance results were taken from the completed project files and local runs. AI assistance did not replace the execution of the project or the collection of its results.